Nile University (NU) IoT Research Internship Report

Day 3

Overview

Day 3 focused on connecting the IoT system to the cloud using the Adafruit IO platform. Both HTTP and MQTT protocols were implemented for cloud communication, after which the code was restructured using inter-process communication (IPC) to create a modular, maintainable system. The day closed with configuring the system to start automatically on boot using systemd services.

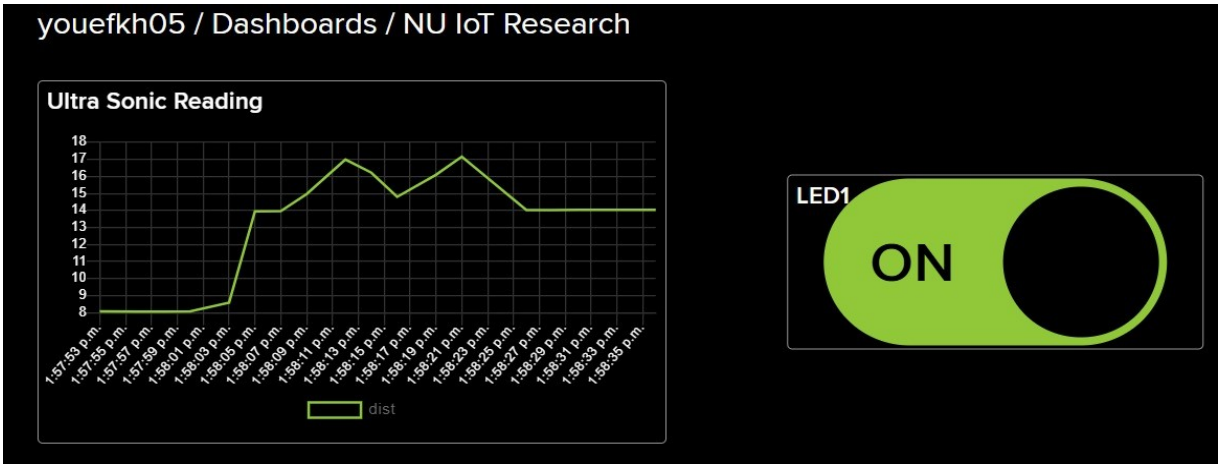
Task 1: Adafruit IO HTTP Communication

Created a Python class to communicate with Adafruit IO using HTTP REST API calls, allowing sensor data to be uploaded to the cloud and devices to be controlled remotely.

Key elements:

- Built an AdafruitIO class wrapper using the requests library for HTTP calls
- Stored credentials securely using environment variables (IO_USERNAME, IO_KEY)
- Implemented a post() method to upload data to a feed and a get() method to retrieve the latest value from a feed
- Used proper HTTP headers for authentication (X-AIO-Key, Content-Type: application/json)

Result: ultrasonic sensor distance readings were uploaded to Adafruit IO every second, and an LED could be controlled remotely by reading the led1 feed and updating the GPIO pin. HTTP was a good starting point because of its simple request/response model, ease of debugging, and suitability for infrequent updates that don't need a persistent connection.



Connection to Day 2: this task took the sensing and data pipeline built on Day 2 (HC-SR04 readings, logged locally) and extended it outward to the cloud, making the data accessible remotely instead of only stored on the device.

Task 2: Adafruit IO MQTT Communication

Replaced the HTTP implementation with MQTT, a lightweight publish/subscribe protocol built specifically for IoT applications.

Key elements:

- Built an AdafruitIO MQTT class wrapper using the paho-mqtt library
- Connected to the io.adafruit.com broker on port 1883
- Published sensor readings to the dist feed and subscribed to the led1 feed for control commands (we used HiveMQ as a broker)
- Registered on_connect() and on_message() callback functions, with on_message() triggering automatically on new data
- Ran the MQTT client in a background thread using loop_start()

Improvement over Task 1: HTTP required repeated request/response calls and polling to check for updates, which is less efficient. MQTT solved this with a persistent connection and a push model, lowering network overhead and enabling real-time bidirectional communication, better suited to IoT systems with limited bandwidth.

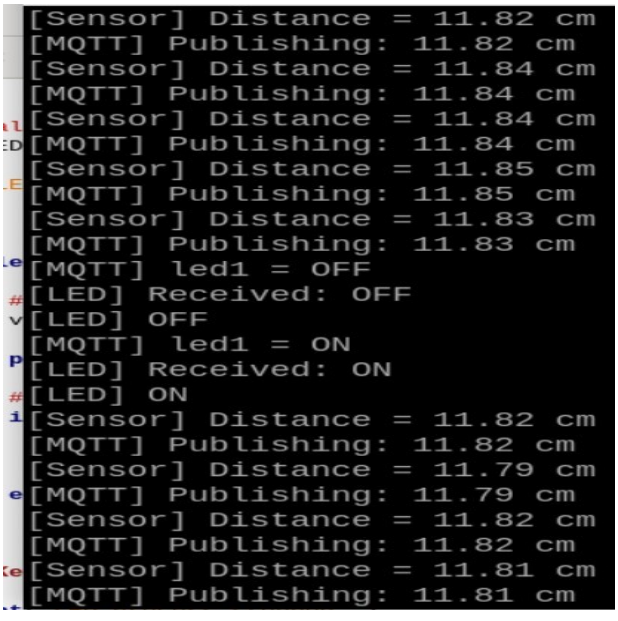
Task 3: Inter-Process Communication (IPC)

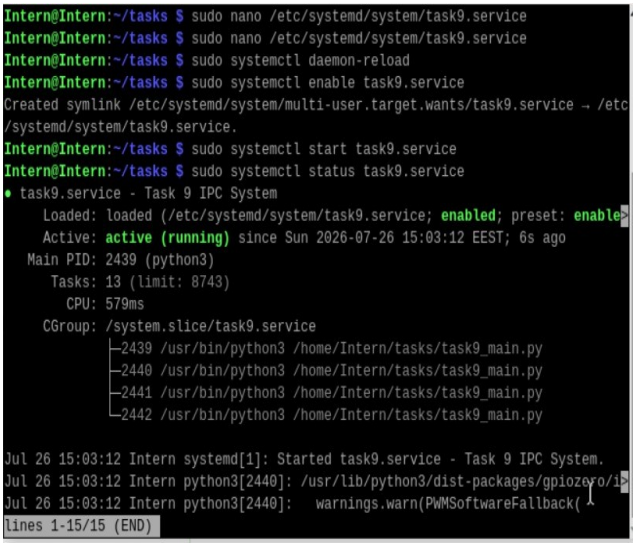
Restructured the code from a single monolithic script into three separate processes communicating through multiprocessing queues:

- Sensor Process (sensor.py): reads the HC-SR04 sensor every second and pushes readings into distance_queue
- MQTT Process (mqtt.py): reads from distance_queue and publishes to Adafruit IO via MQTT, and subscribes to the led1 feed, pushing incoming commands into led_queue
- LED Process (led.py): waits for commands from led_queue and turns the LED on GPIO 17 on or off accordingly
- Main Process (main.py): creates the queues, starts all three processes, and handles graceful shutdown on Ctrl+C

Communication flow: Sensor → MQTT (via distance_queue) → cloud, and cloud → MQTT → LED (via led_queue).

Improvement over Task 2: Task 2 established the communication protocol but still ran as one script handling sensing, cloud communication, and actuation together. This task solved that by separating concerns into independent processes, improving modularity, scalability, maintainability, fault isolation, and resource efficiency across CPU cores.





Task 4: Auto-Start Service Configuration

Created a systemd service (/etc/systemd/system/task9.service) to automatically start the IPC system on boot and restart it if it crashes.

Key configuration:

- [Unit]: description and After=network.target so the service starts only once networking is available
- [Service]: Type=simple, runs as the Intern user, sets the working directory, defines ExecStart to run task9_main.py, Restart=always with a 5 second delay, and credentials passed via Environment variables
- [Install]: WantedBy=multi-user.target

Improvement over Task 3: Task 3 made the system modular but still had to be started manually. This task solved that final gap by making the multi-process system boot automatically, recover from crashes on its own, and log its activity, turning it into a self-sufficient, production-style deployment.

Outcome

- Connected the IoT system to the cloud, first with HTTP and then with the more efficient MQTT protocol
- Understood and compared the trade-offs between HTTP (request/response, polling, simple) and MQTT (publish/subscribe, persistent, real-time)
- Refactored the system into independent, communicating processes using IPC queues
- Deployed the complete system as an auto-starting, self-restarting systemd service
- Completed the progression from a single local script to a modular, cloud-connected, self-managing IoT system